

摘要

对具有前向和后向安全的动态可搜索加密问题进行了研究。

其中最有效的方案的局限性：需要为每个唯一的关键字维护一个操作计数器，这些关键字可以本地存储在客户端，也可以加密存储在服务器上，并在每次操作中无意识地访问（例如，使用无意识的映射）。

作者提出了三种新方案，克服了上述局限性，实现了恒定的永久客户端存储，并在理论和实验上提高了性能。

前两个方案采用了“静态到动态”的转换，消除了在搜索过程中进行无意识访问的需要。

第三个方案是第一个准最优的前向和后向DSE方案，它只有检索查询结果的对数开销（与之前的删除无关）。虽然它确实需要在搜索过程中进行无意识访问，以将永久客户端存储保持在最低限度，但它的实际性能比具有准最优搜索的最佳现有方案高出四个数量级。

1 介绍

背景

可搜索加密（SE）：假设一组加密文档，查询可能是返回包含关键字 w 的文档的所有标识符，而无需首先解密文档。

泄漏：为了实现良好的实际性能，允许SE方案向服务器透露有关数据集的一些信息。可以在初始化阶段（例如，数据集大小）或在执行查询期间（例如，访问和搜索模式、包含 w 的文档的标识符以及之前执行此搜索的时间）发生。

动态可搜索加密

可以有效地支持加密数据集中的修改而无需重新初始化协议。

然而，发展安全的动态可搜索加密方案是比较有挑战性的，因为更新可能会向服务器泄露额外的信息。所以在此提出了两个安全概念：前向安全和后向安全

- 前向安全：确保新的更新不会与之前的操作关联（直到搜索到相关关键字）。

好处：允许“动态”构建加密数据集；减轻依赖于对抗性文件注入的某些泄漏滥用攻击

- 后向安全：确保如果在搜索 w 之前删除了包含关键字 w 的文档，则此搜索的结果不会显示有关此文档的任何信息。

BP-I: 仅显示当前包含 w 的文件的标识符及其存储时间

BP-II: 还显示了 w 的所有先前更新的时间戳和类型（插入/删除）

BP-III: 还显示了它取消的每个先前删除的插入。

挑战1：小型客户端存储的动态可搜索加密

文献中的大多数实用DSE构造都要求客户端在本地存储一个表，该表为数据集中的每个关键字保存一个计数器 a_w ，用于计算 w 的更新次数（一些方案存储了一个额外的计数器用于搜索）。

但是这增加了客户端的存储

在许多情况下，在本地存储每个单词的计数器是有问题的。此外，如果我们想支持从多个设备访问加密数据库的能力，这种方法将特别麻烦，因为它需要它们之间的同步和状态传输。

Using oblivious primitives

把 w 加密后存储在服务器，但这会威胁前向隐私，所以还需要使用OMAP，向服务器隐藏每次访问的单词条目。

缺点：需要对数轮的交互。

挑战2：具有（准）最优搜索的DSE

现有方案用遗忘访问取代了检索结果所需的 n_w 访问，并在最优成本上实现了多对数开销。

然而，他们使用“黑盒”的遗忘原语导致了性能相当差的方案，特别是由于通信成本开销大。

作者的成果

1. 提出了一种从任何隐藏结果的静态SE到向后和向前隐私DSE的黑盒缩减。称为 SD_a

。它有 $O(a_w + \log N)$ 的搜索成本，其中表示关键字 w 的更新总数， $O(\log N)$ 表示摊销的更新成本。最重要的是， SD_a 是第一个具有 $O(1)$ 永久客户端存储而不使用遗忘原语的DSE，因此它大大优于所有现有的搜索方案。

2. SD_a 在进行摊销更新时，其临时客户端存储可能会增长得相当大，达到 $O(N)$ 的级别，这意味着在某些情况下可能会占用大量的存储空间。为了避免这种情况，提出了 SD_d 版本，该版本使用了去摊销更新方法。 SD_d 在搜索开销方面与 SD_a 相同，但在很多场景下，它的性能超越了现有的低客户端存储的DSE方案。
3. 对于删除密集型查询工作负载，提出了QOS，这是一种具有准最优搜索时间 $O(n_w \log i_w)$ 和 $O(1)$ 客户端存储的DSE，在搜索过程中大大优于现有的准最优方案，其中 n_w 表示包含关键字 w 的文件数， i_w 表示 w 的插入次数。事实上，对于较大的删除百分比（约40-80%，取决于部署设置），它优于所有其他方案。

我们所有的构建都是向前和向后安全的（用于 SD_a 和 SD_d 是BP-II，QOS是BP-III）。

TABLE I: Comparison of existing forward-and-backward-private DSE with small client storage. N is an upper bound for total insertions, $|W| = \#$ distinct keywords. For keyword w : $a_w = \#$ updates, $i_w = \#$ insertions, $d_w = \#$ deletions, $n_w = \#$ files containing w . RT is $\#$ roundtrips for retrieving $DB(w)$. BP stands for backward privacy type (the smaller, the better) and *am.* for amortized efficiency. The O notation hides polylogarithmic factors. WO stands for storing search/insertion counters for each w at an oblivious map. To minimize client storage, oblivious map stashes are stored at the server and downloaded every time.

Scheme	Computation		Communication		Search RT	Client Storage	BP
	Search	Update	Search	Update			
MONETA [10]	$\tilde{O}(a_w \log N + \log^3 N)$	$\tilde{O}(\log^2 N)$	$\tilde{O}(a_w \log N + \log^3 N)$	$\tilde{O}(\log^3 N)$	2	$O(1)$	I
WO-MITRA [29]	$O(a_w + \log^2 W)$	$O(\log^2 W)$	$O(a_w + \log^2 W)$	$O(\log^2 W)$	$O(\log W)$	$O(1)$	II
SD_a	$O(a_w + \log N)$	$O(\log N)(am.)$	$O(a_w + \log N)$	$O(\log N)(am.)$	1	$O(1)$	II
SD_d	$O(a_w + \log N)$	$O(\log^3 N)$	$O(a_w + \log N)$	$O(\log^3 N)$	1	$O(1)$	II
ORION [29]	$O(n_w \log^2 N)$	$O(\log^2 N)$	$O(n_w \log^2 N)$	$O(\log^2 N)$	$O(\log N)$	$O(1)$	I
HORUS [29]	$O(n_w \log d_w \log N + \log^2 W)$	$O(\log^2 N)$	$O(n_w \log d_w \log N + \log^2 W)$	$O(\log^2 N)$	$O(\log N)$	$O(1)$	III
QOS	$O(n_w \log i_w + \log^2 W)$	$O(\log^3 N)$	$O(n_w \log i_w + \log^2 W)$	$O(\log^3 N)$	$O(\log W)$	$O(1)$	III

与其他DSE的详细比较可以在表I中看到，其中我们只关注具有小型客户端存储的方案。

表I中的所有方案都使用OMAP， SD_a 除外；它们可以通过在服务器上存储存储并使用PRF生成密钥来实现 $O(1)$ 存储。

技术概述

SD_a 和 SD_d 利用经典技术将静态数据结构转换为动态数据结构它们将 N 次更新的结果存储在 $\log N$ 个单独索引的序列中。每次更新首先存储在最小的索引中，每当存在两个大小相同的索引时，客户端就会下载并合并到一个新的索引中。

隐私延迟重建

对于 SD_d ，我们提出了一种实现[46]中用于可搜索加密的延迟重建去摊销技术的方法。

每次更新都需要运行底层静态SE设置例程的固定少量步骤。

为了实现前向和后向安全，静态SE必须具有一个自然“可分解”为具有小的、可有效检索的局部状态的步骤的设置。

OMAP用于更新；不是用于搜索

尽管 SD_d 使用了OMAP，但它只在更新期间被访问。因此，搜索是以非交互方式执行的。

QOS：实用的准最优搜索

与之前需要执行 n_w 不经意访问的所有准最优搜索DSE不同，QOS需要一个OMAP查询，这使得它在实践中更快。这是通过将结果检索与遗忘基元解耦来实现的。

QOS的插入和大多数现有的DSE方案差不多，有差别的地方在于它处理删除的方法。对于每个 w ，客户端维护一个描述其更新历史的“概念性”二叉树

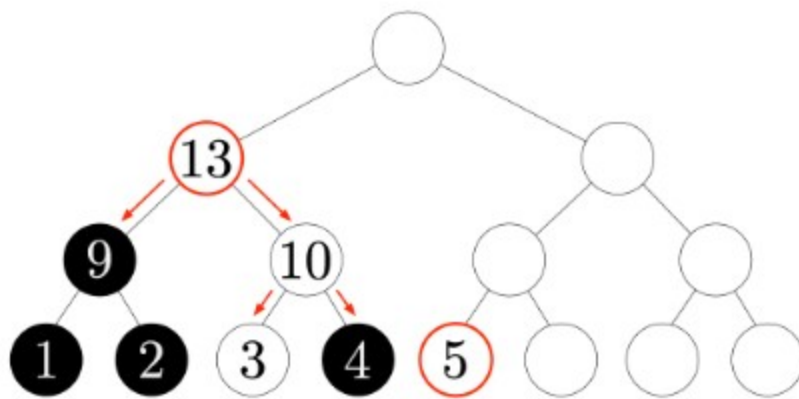


Fig. 5: Update tree for QOS with maximum insertions $N = 8$. Nodes are labeled with $[1, 15]$ leaf-to-root and left-to-right. This is the tree state after five insertions 1-5, and three deletions for 1, 2, 4. A subsequent search starts from the Best Range Cover of leaves $[1, 5] = (13, 5)$ and proceeds downwards until it finds a black node or a leaf. The result is (3, 5).

在实践中，这是通过将第 $\mathcal{I}$ 个插入 (w_i, id_i) 存储到 $\mathcal{I}$ 中取决于叶子标签 i 的位置来实现的。

清理

许多DSE方案（如 SD_a 和 SD_d ）在删除时仍然保留条目，因此服务器存储在删除后并未减少。这在存储加密条目的哈希表中几乎是不可避免的，尽管有前向安全的DSE方案存在，但尚无后向安全的方案具备相同特性。此外，未来对减少搜索的时间的研究被认为

为比节省服务器存储空间更为重要。许多现有DSE方案都有“清理”阶段，通常与搜索同时执行。 SD_a 和 SD_d 方案可以在更新过程中实现这种清理过程（ SD_a 比较自然，而 SD_d 需要额外的记录）。而类最优方案如QOS由于搜索几乎不受删除影响，天然地具备了这一点。

实验评估

分别对比 MITRA 和 HORUS

SD_a 和 SD_d 比MITRA快34倍和20倍，具有非交互式的额外好处。

QOS远远优于HORUS（在我们的实验中为4-16531倍）。

方案在更新中表现较差，所有结果均基于10%的删除条目比例。然而对于删除比例更高的情况，QOS显示出成为最有效解决方案的潜力，在删除比例在40%到80%之间时，QOS的性能优于MITRA和 SD_d ，具体取决于插入数量。

相关工作

动态可搜索加密（Dynamic Searchable Encryption, DSE）领域的相关研究，包括前向安全和后向安全的研究成果，并总结了不同方案在性能和安全性方面的差异：

1. 早期工作和安全性改进：可搜索加密最初由 Song 等人提出，之后 Curtmola 等人对其安全性进行了改进，引入了广泛使用的安全模型。早期的 DSE 方案多为静态设计，无法支持数据集的动态更新，随着需求增加，动态可搜索加密逐渐成为研究重点。
2. 前向安全和后向安全：为确保数据的安全性，研究者提出了前向安全（确保新插入的操作不会泄露与先前操作的关系）和后向安全（确保已删除的数据在后续查询中不会泄露）的概念。作者提到了这些概念的发展历程，如 Stefanov 等人和 Bost 等人对于后向安全的研究和分类。
3. 现有方案的局限性：许多现有的 DSE 方案需要客户端存储大量的本地数据，例如为每个关键词维护操作计数器，或者依赖服务器上的复杂加密结构，这可能导致性能问题。为了解决这些问题，一些研究提出了基于隐私保护技术（如 ORAM）的优化方案，但这些方案在实际应用中仍存在性能瓶颈。
4. QOS方案的创新：在前人的基础上，QOS 提出了通过维护“概念二叉树”来优化更新历史的存储和查询效率。相比其他方案，QOS 在客户端存储要求、查询效率和安全性方面均实现了改进。

ORAM的最新进展

最近，文献[47]和[6]提出了新颖的ORAM (Oblivious RAM) 构造，效率得到了改善，与理论最优开销 $O(\log N)$ 相匹配。这些构造可以被修改为产生“无感知映射”，从而在渐近上提升我们的SDd和QOS方案，以及文献[27]、[10]和[29]的性能。然而，这些改进方案的性能是按摊销计算的。尽管有可能对其进行去摊销，但这无疑会导致额外的成本。

预备知识

伪随机函数

- $Gen(1^\lambda) \in \{0, 1\}^\lambda$: 生成密钥的函数
- $F : \{0, 1\}^\lambda \times \{0, 1\}^l \rightarrow \{0, 1\}^{l'}$: 伪随机函数 (PRF) 族
- $R : \{0, 1\}^l \rightarrow \{0, 1\}^{l'}$: 真随机函数

F 是安全的PRF如果：对于所有PPT(probabilistic polynomial-time)的攻击者有

$$Adv[Pr[K \leftarrow Gen(1^\lambda); Adv^{F(K, \cdot)}(1^\lambda) = 1] - Pr[Adv^{R(\cdot)}(1^\lambda) = 1]] \leq v(\lambda)$$

可搜索加密

$$\Sigma = (Setup, Search, Update)$$

- $(K, \sigma, EDB) \leftarrow Setup(\lambda)$

K : 客户端的密钥

σ : 客户端的本地状态

EDB : 是发送到服务器的（最初为空）加密数据库。

$Setup(\lambda, N)$: 一个设置过程，该过程采用参数 N 表示支持的最大条目数。

- $DB(w) \leftarrow Search(K, q, \sigma; EDB)$

一种搜索数据库的协议。我们考虑单个关键字的搜索查询，即 $q = w \in \Lambda^*$

- $Update(K, op, w, id, \sigma; EDB)$

$op = add/del$

Σ 的安全性由泄露函数 $\mathcal{L} = (\mathcal{L}^{Stp}, \mathcal{L}^{Srch}, \mathcal{L}^{Updt})$ 参数化，泄露函数描述了在整个协议

执行过程中向服务器显示的信息。

文献中的标准搜索泄露类型包括搜索模式：揭示了哪些搜索与同一 w 相关；访问模式：在搜索 w 的过程中揭示了 $DB(w)$ 。注意，如果客户端希望检索实际文件而不仅仅是它们的标识符（除非文件本身以受保护的方式存储），那么访问模式是无法避免的。

result hiding (结果隐藏)：避免这种泄漏的方案

具有泄漏 $\mathcal{L}$ 的安全SSE方案除了此泄漏之外，不应透露任何关于数据库DB的信息。

- **Definition 1:** DSE方案 Σ 相对于泄漏函数 $\mathcal{L}$ 是自适应安全的，对于任何PPT对手 Adv 发出 $poly(\lambda)$ 查询/更新 q ，存在一个有状态的PPT模拟器 $Sim = (SimInit, SimSearch, SimUpdate)$ ，使得

$$|Pr[Real_{Adv}^{SSE}(\lambda, q) = 1] - Pr[Ideal_{Adv, Sim, \mathcal{L}}^{SSE}(\lambda, q) = 1]| \leq v(\lambda)$$

前向安全和后向安全

如果发生新更新时无法将其连接到以前的操作，则该方案为前向安全。例如，应该无法分辨添加是针对新关键字还是之前搜索过的关键字。

- **Definition 2:** 如果更新泄漏函数 $\mathcal{L}^{Updt}$ 可以写成： $\mathcal{L}^{Updt}(op, w, id) = \mathcal{L}'^{Updt}(op, id)$ ，其中 $\mathcal{L}'$ 是无状态函数， $op = add/del$ ， id 是文件标识符，则支持单个关键字添加/删除的 $\mathcal{L}$ 自适应安全DSE方案是前向安全的。

后向安全DSE方案限制了服务器在搜索 w 期间学习的信息，其中一些条目之前已被删除。理想情况下，该方案不应透露这些已删除条目的任何信息，至少不应透露其相应的文件标识符[52]。

Q 是一个列表，每个操作有一个条目。对于搜索，条目是 (u, w) ，其中 u 是时间戳， w 是搜索关键字。对于更新，它是 $(u, op, (w, id))$ ，其中 $op = add/del$ ， id 是修改后的文件。

- $TimeDB(w) = \{(u, id) | (u, add, (w, id)) \in Q \wedge \forall u', (u', del, (w, id)) \notin Q\}$

是返回已添加到数据库中但未删除的关键字 w 的所有时间戳文件标识符对的函数

- $Updates(w) = \{u | (u, add, (w, id)) \in Q \text{ or } (u, del, (w, id)) \in Q\}$

是返回 w 的每次插入/删除操作的时间戳的函数

- $DelHist(w) = \{(u^{add}, u^{del}) | \exists id : (u^{add}, add, (w, id)) \in Q \wedge$

$$(u^{del}, del, (w, id)) \in Q\}$$

是一个函数，它为每个删除时间戳返回它取消的相应插入的时间戳。

- **Definition 3** : L自适应安全SSE方案具有后向安全：

BP-I (BP with insertion pattern): iff $\mathcal{L}^{U_{pdt}}(op, w, id) = \mathcal{L}'(op)$ and $\mathcal{L}^{Srch}(w) = \mathcal{L}''(\text{TimeDB}(w), a_w)$

BP-II(BP with update pattern): iff $\mathcal{L}^{U_{pdt}}(op, w, id) = \mathcal{L}'(op, w)$ and $\mathcal{L}^{Srch}(w) = \mathcal{L}''(\text{TimeDB}(w), \text{Updates}(w))$

BP-III (weak BP): iff $\mathcal{L}^{U_{pdt}}(op, w, id) = \mathcal{L}'(op, w)$ and $\mathcal{L}^{Srch}(w) = \mathcal{L}''(\text{TimeDB}(w), \text{DelHist}(w))$

其中 $\mathcal{L}'$ 和 $\mathcal{L}''$ 是无状态函数，我们强调，由于 $\text{TimeDB}(w)$ ，上述定义（甚至是BP-I）揭示了当前包含 w 的文件——这是为了解释检索实际文件时的泄漏。

具有最佳搜索时间的DSE

大多数现有的DSE方案采用将删除作为常规条目“存储”的方法。在搜索过程中，它们用于过滤出哪些插入条目已被删除。这种方法意味着搜索成本将为 $\Omega(a_w)$ ，即 w 的总更新次数与最优成本 $O(n_w)$ 呈线性关系，而当前包含 w 的文件数量与最优成本呈线性关系。

- **Definition 4**: 如果搜索的渐近复杂度为 $O(n_w)$ （或 $O(n_w) \cdot \text{polylog}(N)$ ），则DSE方案 Σ 具有最优（或准最优）搜索时间。

从静态方案到动态方案

摊销结构

这个方案描述了一种动态可搜索加密（Dynamic Searchable Encryption, DSE）构造方法，称为 SD_a ，它以一个静态结果隐藏的可搜索加密（SE）方案为起点，并在其中引入了更新操作。这里，每个更新操作都表示为三元组 (w, id, op) ，其中 w 为关键词， id 为文档标识符， op 为添加或删除操作。系统通过一系列独立的加密索引 $(EDB_0, \dots, EDB_{\log(N-1)})$ 来存储更新，每个索引大小逐渐增加。对于每次更新，客户端找到第一个为空的索引位置，将之前所有的非空索引解密并合并，然后连同当前更新一起创建新的索引，再存储在新的位置上。

$(K, \sigma; EDB) \leftrightarrow \text{Update}(K, op, w, id, \sigma; EDB)$

Server:

- 1: Find the minimum j such that $EDB_j = \emptyset$
- 2: Send to client $EDB_0, \dots, EDB_{j-1}$

Client:

- 3: Set $A \leftarrow \emptyset$
- 4: **for** $i = 0, \dots, j - 1$ **do**
- 5: $A \leftarrow A \cup \text{SE.DecryptAll}(K[i], \sigma[i], EDB_i)$
- 6: $K[i] \leftarrow \perp, \sigma[i] \leftarrow \perp$
- 7: $(K[j], \sigma[j], EDB_A) \leftarrow \text{SE.Setup}(1^\lambda, A \cup (w, id, op))$
- 8: Send EDB_A to server

Server:

- 9: Set $EDB_j \leftarrow EDB_A$
- 10: **for** $i = 0, \dots, j - 1$ **do**
- 11: Set $EDB_i \leftarrow \emptyset$

在查询过程中，客户端依次对每个非空索引进行搜索，解密结果并提取需要的文档标识符和操作类型，并过滤掉删除操作的条目，从而得到最终的搜索结果。

$DB(w) \leftrightarrow \text{Search}(K, q, \sigma; EDB)$

Client $\leftrightarrow$ Server:

- 1: $\mathcal{X} \leftarrow \emptyset$.
- 2: **for all** i such that $EDB_i \neq \emptyset$ **do**
- 3: Let $\mathcal{X}_i \leftrightarrow \text{SE.Search}(K[i], q, \sigma[i]; EDB_i)$
- 4: $\mathcal{X} \leftarrow \mathcal{X} \cup \mathcal{X}_i$

Client:

- 5: Decrypt entries of $\mathcal{X}$ with K and parse them as (id, op)
- 6: $DB(w) \leftarrow \{id \mid (id, add) \in \mathcal{X} \wedge (id, del) \notin \mathcal{X}\}$

安全性

1. **前向隐私**：由于每次更新都会使用新的密钥运行SE.Setup，且加密索引的大小仅依赖于之前的更新数量，因此可以通过SE的设置模拟器（setup simulator）来模拟更新操作。即使SE的setup泄漏也只是泄露数据库大小，这意味着服务器在更新过程中无法获得与之前条目（包括关键词 w 相关的条目）有关的信息，就是说看到的信息独立于EDB中的任何先前条目，从而实现了前向隐私。

2. **后向隐私**：首先，SE是结果隐藏的，并且删除操作也作为普通条目存储，因此服务器无法知道哪些文件之前包含关键词 w 。在搜索过程中，服务器仅能知道关键词在数据库中匹配条目的数量 $|DB(w)|$ 以及各个加密索引（EDB）的搜索结果数量。为了模拟这个过程，只需知道关键词 w 的每次更新时刻和当前的总更新计数，这些信息足以确定每次更新对应的索引位置。基于这些信息，方案达到了BP-II级别的后向隐私。

此外，*SimSearch*并不总是需要完整的更新信息来模拟搜索记录，只需要知道每次更新应映射到的索引即可。在某些情况下（例如更新计数为2的幂时），实际的泄露信息可能更少，因为更新记录可能集中在同一个索引中。

效率

- **服务器存储**：在完成 N 次更新后， SD_a 包含 $\log N$ 个加密索引，每个要么为空，要么存储 2^i 个条目。如果底层SE方案的存储是线性的，那么SDa的服务器存储复杂度为 $O(N)$ 。
- **查询开销**：假设SE的搜索时间是最优的，那么检索关键词 w 的所有更新的开销是 $O(a_w)$ ，其中 a_w 是关键词 w 的匹配条目数。由于最多存在 $\log N$ 个非空索引EDB，每个都需要进行一次搜索，因此SDa的总搜索时间为 $O(a_w + \log N)$
- **更新的均摊成本**：在完成 2^j 次更新后，客户端会对不同规模的索引多次运行 SE.Setup。从规模 2^j 到规模1的索引逐次递减，且每个规模的次数递增。假设底层静态方案的设置时间为线性，那么在完成 N 次更新后的均摊更新成本为 $O(\log N)$
- **均摊成本**：由于每层索引的大小按指数增长，因此在总更新次数为 N 时，只有 $\log N$ 层索引。均摊之后，每次更新的成本仅为 $O(\log N)$ 而不是 $O(N)$ ，因为更新操作只是偶尔需要重构高层索引，其他大部分时间的操作仅限于低层索引。
- **密钥存储优化**：在PiBas方案下，客户端需要为每个加密索引实例存储一个密钥，总共需存储 $O(\log N)$ 个密钥。为了将本地存储缩减至 $O(1)$ ，可以通过伪随机函数（PRF）使用单个主密钥来生成每个索引的密钥。
- **通信轮次**：在使用PiBas的 SD_a 方案中，查询关键词 w 的结果仅需单轮通信。更新操作则需要两次通信：一次用于获取旧的索引进行合并，另一次用于客户端向服务器写入新的索引 EDB_j ，这第二次通信可以合并到下一次操作中。
- **删除条目的清理**： SD_a 方案易于清理已删除的条目。在更新时，客户端会先检查并移除旧索引（ $EDB_i, i < j$ ）中的对应删除条目，并用虚假记录填充，使得能将新数据存储到新索引 EDB_j 。

去摊销结构

SD_a 具有出色的性能，但是有可能这个索引偶尔会变得非常大。为了克服这个障碍，我们介绍 SD_a 构造的去摊销版本，我们称之为 SD_d 。

SD_d 为每个 $i = 0, \dots, \log N$ 维护四个加密索引 $OLDEST_i$, $OLDER_i$, OLD_i 和 NEW_i , 每个“旧”索引要么为空，要么恰好包含 2^i 个项。

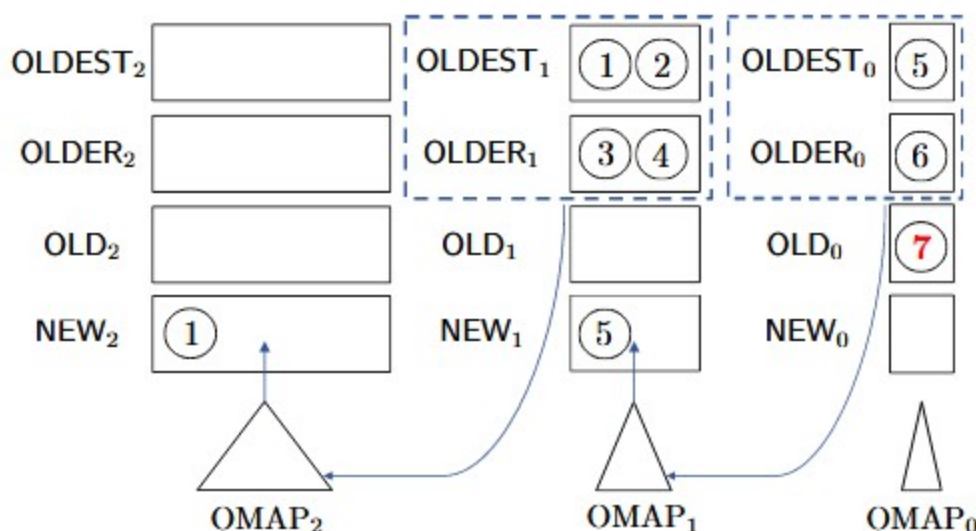
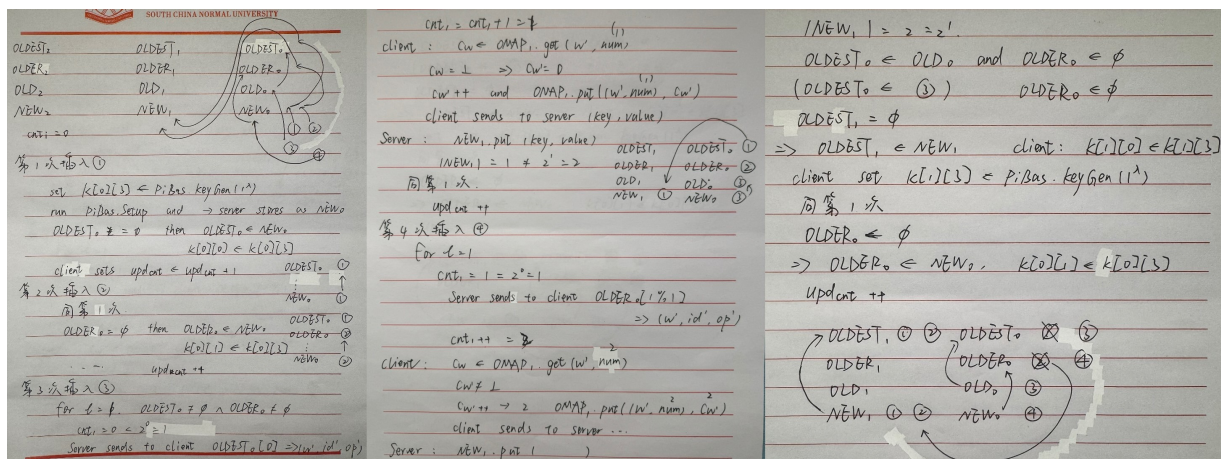


Fig. 3: SD_d : from PiBas to DSE (de-amortized version). These are the encrypted indexes after 7 consecutive insertions 1 – 7. Each level i contains 3 searchable encrypted indexes (OLDEST, OLDER, OLD) and one index (NEW) that is used for merging and rebuilding into a single index the OLDEST and OLDER indexes of the previous level $i - 1$. The update algorithm passes through all levels and moves one element using $OMAP_i$ from level $i - 1$ to level i . Searching for a keyword w requires to search all the OLDEST, OLDER and OLD encrypted indexes and return to the client all the individual search results.

当 SD_d 与摊销结构 SD_a 的差别就在更新期间

具体而言，更新算法从最大到最小遍历所有大小 i ，并将一个元素从 $i-1$ 级索引集移动到 i 级索引集。对于每个层级，如果存在非空的 $OLDEST_{i-1}$ 和 $OLDER_{i-1}$ ，说明在接下来的 2^i 次更新中，必须完整重建大小为 2^i 的索引，以免在层级 $i-1$ 处耗尽空间。因此，在每次 2^i 次更新中，都需要执行一次 $PiBas.Setup$ ，将一个元素 (w, id, op) 从 $OLDEST_{i-1} \cup OLDER_{i-1}$ 移动到 NEW_i 。同时，为了确保前向隐私，需要保证这一过程不会向服务器泄露任何信息。



- 映射和更新:** 每个PiBas实例的EDB加密索引包含一个映射 T , 其键值通过PiBas.Map函数计算。客户端在进行 2^i 次更新时, 从服务器顺序检索并解密与 $OLDEST_{i-1}$ 和 $OLDER_{i-1}$ 相关的条目, 利用当前的全局更新计数器高效计算下一个条目的位置。
- 计数器的处理:** Map算法的输入包括密钥 K 、条目 w 、 id 、 op 和计数器 c 。由于 c 无法高效地存储在本地, 因此为每个大小为 2^i 的索引使用一个隐私映射 $OMAP_i$, 将 w 映射到 c 。在每一步中, 客户端查询 $OMAP_i$, 使用得出的 c 进行Map操作, 递增 c 并存回 $OMAP_i$ 。此外, c 在 NEW_i 重建之间需要重置, 因为PiBas搜索始终从零开始。这通过进一步将 $OMAP_i$ 设计为映射 (w, num) 至 c 来实现, 其中 num 表示 NEW_i 之前重建的次数。
- 索引的移动与更新:** 当 NEW_i 完全构建 (大小为 2^i) 时, 服务器将其移动到 $OLDEST_i$ 、 $OLDER_i$ 和 OLD_i 中的最旧非空索引, 并删除 $OLDEST_{i-1}$ 和 $OLDER_{i-1}$ 。若 OLD_{i-1} 存在, 服务器会将其移动到 $OLDEST_{i-1}$ 。每次更新还会在可用的最旧位置为新插入的条目创建一个大小为0的“单例”加密索引。所有PiBas实例在初始化时都会选择一个新的密钥。

安全性

由于搜索协议基本相同, 因此 SD_d 的后向安全被证明与 SD_a 完全相同。

前向隐私是通过以下几个方面实现的:

- 新的PRF评估:** 每次更新 (w, id, op) 时, 服务器会看到新的伪随机函数 (PRF) 评估。这是因为每个PiBas实例都选择新的密钥, 这样每次更新都会产生新的密钥, 从而加强了隐私保护。
- 响应隐藏:** PiBas的修改版本是响应隐藏的。这意味着服务器无法根据客户端的请求或更新的响应来推断出任何有关已访问或更新条目的信息, 从而进一步保护了隐私。

3. **受限的数据访问模式**：每次更新访问的都是预先确定的位置，并且在最多需要 $2 \cdot \log N$ 个映射数据结构中进行操作。这种使用方式限缩了访问模式，使得服务器无法推测出特定条目的内容。同时，执行的读/写操作通过使用隐私映射（oblivious map）进行处理，这种查询方式不会向服务器泄露有关访问条目的任何信息。

性能分析

更新需要 $O(\log N)$ 次 OMAP 查询，总访问开销为 $O(\log^3 N)$ ，搜索时间为 $O(a_w + \log N)$ ，服务器存储与总更新次数呈线性关系

至关重要的是，客户端存储为 $\tilde{O}(\log^2 N)$ ，用于存储所有 OMAP 存储和 PiBas 密钥。如果我们想最小化客户端存储，还有两个额外的技巧：（i）存储在服务器上，并在更新过程中根据需要下载，而不影响更新渐近线，以及（ii）PiBas 的密钥是由单个主密钥通过 PRF 生成的。这些使客户端存储为 $O(1)$ 。

具有准最优搜索的高效 DSE

与现有的方案如 Orion 和 Horus 不同，后者在检索结果时需要多次使用隐私访问，QOS 仅在搜索过程中需要对隐私映射进行一次读写。检索结果的其余操作则在服务器端通过一个树状数据结构来识别已删除的条目。

QOS 的核心概念是一个拥有 N 个叶子的完全二叉树，其中 N 是数据结构中插入（和删除）的总数的上限。函数 $label(v)$ 返回一个在 $[1, 2N - 1]$ 中的值，是树节点“自然”标记的结果：每个叶子从 1 到 N 被标记，内部节点则按层级顺序递增标记。所有节点最初都是白色的，客户端为每个关键字维护一个概念性的树结构。在所述树中，插入的条目对应于从左到右填充的叶子。

当一个条目被删除时，对应的叶子节点被标记为黑色，其祖先的颜色也可能根据规则更新：如果一个节点的两个子节点都是黑色，则该节点也被标记为黑色。这种方式有效地消除了整个子树在搜索时的考虑。

为了在 i_w 次插入后检索结果，算法首先计算出标记为 1 到 i_w 的叶子的最佳范围覆盖，然后从这些节点向下搜索。遇到黑色节点时搜索停止，表明以下没有未删除的条目。到达的非黑色叶子会被添加到结果集中。分析表明，虽然覆盖这些叶子的子树大小小于 $2i_w$ ，但在搜索过程中访问的节点数是 $O(n_w \log i_w)$ 。

方案描述

Setup

在设置过程中，客户端初始化三个容量分别为 $|W|$, N , N 的空OMAP:

$$1. OM_{cnt}(w) \rightarrow (cnt_w, i_w)$$

cnt_w 是以前搜索 w 的次数, i_w 是以前插入 w 的次数。

$$2. OM_{del}(w, id) \rightarrow label(v)$$

其中 v 是即将插入的叶子节点；在删除过程中，这用于检索要删除的条目的“位置”。

$$3. OM_{state}(w, label(v)) \rightarrow the\ color\ of\ node\ v$$

服务器上存储的加密索引 EDB 包含了盲映射 (oblivious maps) 以及两个空的映射 $\mathcal{I}$ 和 $\mathcal{D}$ ，容量分别为 N 和 D (D 是可删除的上限，也可以作为 OM_{state} 的容量，通常可以设置为 $O(N)$)。

客户端在本地存储这三个盲映射的状态、两个伪随机函数 (PRF) 密钥 $k_{\mathcal{I}}$ 和 $k_{\mathcal{D}}$ ，以及一个对称加密密钥 k 。

Update

客户端首先通过 OM_{cnt} 检索 cnt_w 和 i_w ，然后：

对于更新来说，客户端先把 i_w 的值 $+1$ ，然后写入 OM_{cnt} ；同时还会把这个条目写入 OM_{del} 映射中，其中映射的结果 $label(v)$ 存储在表 $\mathcal{I}$ 中。随后，客户端加密 $id, i_w + 1$ ，得到的结果存储在服务器的映射 $\mathcal{I}$ 中，其位置由哈希函数使用客户端用 $k_{\mathcal{I}}$ 对 $(w, i_w + 1)$ 进行伪随机计算得出的令牌 tk 得出

对于删除来说，在删除操作中，客户端首先通过 OM_{del} 检索存储在树叶位置 pos 的标签。接着，他计算 pos 的所有祖先及其兄弟节点的标签，并从 OM_{state} 中获取它们的颜色。根据这些信息，客户端更新 pos 所有祖先的颜色（在最简单的情况下， pos 被设置为黑色，通常这次删除可能还会导致一些祖先节点变为黑色）。最后，客户端找到在删除过程中首次被设置为黑色的 pos 的最远祖先 d_j 。然后，他在服务器的 $\mathcal{D}$ 中的某个位置（由哈希函数 H' 计算）标记一个条目，同时使用客户端基于 $k_{\mathcal{D}}$ 为 (w, d_j) 计算的伪随机令牌 tk ，并将其新颜色存储在 OM_{state} 中。

Search

客户端首先通过 OM_{cnt} 检索 cnt_w 和 i_w ，然后为其分别计算搜索令牌：

1. 由 k_I 计算 tk_I ，用于检索结果
2. 由 k_D 计算 tk_D ，将用于识别搜索过程中遇到的与删除相对应的黑色节点。

随后客户端将这些令牌发送到服务器，同时客户端也会增加 cnt_w 的值并将其存储到 OM_{cnt} 中。

服务器先通过递归树结构来计算构成叶节点 $[0, i_w]$ 最佳范围覆盖的树节点 $d_0, \dots, d_m$ 的集合：对每个结点 d_i ，服务器检查表 $\mathcal{D}$ 中 $(H'(tk_D, d))$ 位置是否已被标记，若有则是“黑”节点，否则继续遍历其子节点。当遍历至非黑叶节点时，从加密数据库 $\mathcal{I}$ 中返回相应的条目 $DB(w)$ ，当然了，其位置还是需要通过 $H(tk_I, d)$ 计算。

服务器将检索的结果发送回客户端。客户端通过解密这些值来计算出 $DB(w)$ 。然后客户端使用新的伪随机令牌和增加的搜索计数器对 $\mathcal{I}$ 和 $\mathcal{D}$ 中的所有条目进行重新映射，然后发送给服务器重新存储。

安全性分析

前向安全：在更新过程中，服务器观察到两种访问：

1. 固定数量的盲映射操作（取决于更新的类型），这些操作什么也不显示
2. 一对 $(key, value)$ ，由建模为随机神谕的哈希函数的输出和语义安全的密文组成。

语义安全显然不会泄露什么信息；

而对于前者，注意到我们从来不会将相同的输入传递给随机神谕两次，这是因为插入过程中会进行 $i_w = i_w + 1$ ，以及删除操作不会对同一节点执行两次。

由于随机神谕的输入依赖于服务器未知的 PRF 令牌，因此向神谕查询以前没有见过的“有效”值是不可行的。而且，每次搜索后，搜索令牌都会更改。

后向安全：

服务器在搜索过程中能知道PRF令牌，从而能够计算出它需要在表中所要访问的数据的位置。这就能让他回忆起这些条目是什么时候生成的，这样就知道了关键字 w 的所有更新操作的时间戳和类型。

同时，服务器知道树的拓扑结构，树的叶子自然地映射到插入的时间戳，服务器可以准确地推断出哪个删除取消了哪个先前的插入。

因此QOS只能实现BP-III安全

效率

使用QOS进行更新需要 $O(\log N)$ 个OMAP查询，总共需要 $O(\log^3 N)$ 的操作和 $O(\log N)$ 次往返，使用参考文献[55]中的OMAP。设置过程与插入和删除的上限 N 和 D 成线性关系，服务器的存储也是如此。搜索时间的计算如下：OMAP查询需要 $O(\log^2 |W|)$ 的操作，最佳范围覆盖的计算需要 $O(\log i_w)$ ，与覆盖大小相同。解析树以检索结果需要 $O(n_w \log i_w)$ 的时间，因为会访问 n_w 个叶子节点，并且从每个叶子节点到最佳范围覆盖中的节点的最大高度为 $\log i_w$ 。即使在遍历过程中每个节点都有一个黑色兄弟节点（这是一个极大的高估），遇到的黑色节点总数也是 $O(n_w \log i_w)$ 。综上所述，使用QOS的总搜索开销为 $O(n_w \log i_w + \log^2 |W|)$ ，并且需要 $O(\log |W|)$ 次交互。

实验评价

在文章的第五部分中，作者进行了实验评估，以验证所提出方案的性能和实际适用性。主要内容包括对比新的DSE（Dynamic Searchable Encryption）方案——如SDa、SDd、QOS——与现有方案的性能，特别是在搜索、更新效率和客户端存储等方面的改进。

评估结果表明，这些新方案在多种场景下性能显著提升。例如，SDa和SDd方案在搜索速度上有显著优势，且在客户端存储要求上最小。对于删除频繁的应用场景，QOS方案在搜索效率上优于其他准最优方案，同时保持较小的客户端存储。这些评估证实了新方案在保证前向和后向隐私的同时，实现了更优的性能。

总结

本文重新审视了前向和后向私密的动态搜索加密（DSE）问题。之前的先进方案要么要求客户端为每个关键词存储计数器，要么在服务器上无意识地访问这些信息，限制了其在实际应用中的可行性。我们提出了三种新方案，具有恒定的客户端存储需求，并在搜索性能上（无论是渐进的还是实验的）优于之前的工作。特别是，我们的两个方案 SD_a 和 SD_d 不仅消除了搜索过程中的无意识访问需求，还最小化了所需的往返次数。 SD_a 的主要缺点是在更新时具有 $O(\log N)$ 的摊销更新成本，导致某些更新成本较低（如 $O(1)$ ）而其他更新成本较高（如 $O(N)$ ）。为了解决这个问题，我们提出了 SD_d ，它提供了去摊销的更新成本。对于小比例删除， SD_a 和 SD_d 在搜索性能上（渐进和实验）均优于之前的MITRA方案。然而，以上方案的主要问题是搜索时间依赖于先前删除的文档，最坏情况下可能导致 $O(N)$ 的搜索开销。因此，对于删除密集的查询工作负载，我们提出了QOS，它是最有效的DSE，具有准最优的搜索时间，显著改善了之前准

最优DSE方案的性能。在许多实际场景中，我们认为SDd会比QOS更好，但QOS在实现最优搜索性能方面迈出了重要一步，因为它与最优搜索时间相比仅有对数级的开销。实验表明，对于超过80%的删除比例（假设的删除密集工作负载），QOS成为更好的选择。未来的一个研究方向是开发具有最优搜索时间的前向和后向私密方案，理想情况下不使用无意识原语且客户端存储需求较小。